IOGraphNet: Graph Neural Networks for Multi-Label I/O Bottleneck Detection in HPC Systems

Anonymous Author(s)

Abstract

I/O performance remains a critical bottleneck in high-performance computing (HPC), often consuming a significant fraction of application runtime for data-intensive workloads. Existing I/O diagnosis tools either rely on manually crafted rules that may miss complex patterns, or use machine learning with flat feature representations that ignore the *topology* of resource sharing between files and processes. We present IOGraphNet, a graph neural network (GNN) approach for automatic I/O bottleneck detection that formalizes the problem as **Topological Resource Contention**—modeling how ranks share files reveals bottleneck patterns even from aggregated Darshan logs without fine-grained traces. Our key insight is that I/O bottlenecks emerge from structural relationships: an N-to-1 star pattern (N ranks writing to 1 file) indicates contention; a 1-to-N pattern indicates metadata stress. Unlike flat-feature models that see "Rank 0 is slow" and "File A is slow" as independent features, IOGraphNet captures that Rank 0's slowness is *caused by* its connection to high-degree File A. Using a Graph Attention Network (GAT) architecture, IOGraphNet performs multi-label classification to detect 8 bottleneck categories, with attention weights providing interpretable *root-cause localization*—identifying exactly which file-rank relationships cause each detected issue. To address the distribution shift between controlled benchmarks and production workloads, we employ a two-stage **self-supervised pre-training** strategy: the model first learns the structural patterns of real HPC I/O by reconstructing masked edges from 1.37 million unlabeled Polaris logs, then fine-tunes on 2,000+ labeled benchmark logs for classification. Our results show that IOGraphNet achieves [X]% Micro-F1 score, outperforming XGBoost and MLP baselines, with ablation studies confirming that the graph topology contributes [W]% improvement over equivalent models without structural edges.

CCS Concepts

• **Computer systems organization** → **High-performance computing**; • **Computing methodologies** → *Machine learning approaches.*

Keywords

I/O performance, bottleneck detection, graph neural networks, HPC, Darshan

1 Introduction

Input/output (I/O) performance is a persistent challenge in high-performance computing (HPC) systems. As computational capabilities continue to scale, the storage subsystem often becomes the limiting factor for data-intensive scientific applications [24]. Studies have shown that I/O can consume up to 90% of total runtime for certain workloads [6], and the gap between compute and storage performance continues to widen. On leadership-class systems, even modest improvements in I/O efficiency can save thousands of node-hours and accelerate scientific discovery.

1.1 Motivation

Diagnosing I/O performance problems requires expertise in the complex HPC I/O stack, which spans applications, high-level I/O libraries (HDF5, netCDF), MPI-IO, POSIX, and parallel file systems (Lustre, GPFS). Domain scientists often lack this expertise, relying on I/O specialists to analyze performance traces—a process that does not scale with the growing number of HPC users and applications.

Automated I/O diagnosis tools have emerged to address this challenge. Darshan [7] provides lightweight profiling to capture I/O behavior, generating logs with hundreds of counters per file access. Tools like Drishti [2] apply rule-based heuristics to identify common patterns, while AIIO [10] uses machine learning (ML) models such as XGBoost to classify bottlenecks automatically. More recently, large language models (LLMs) have been explored for I/O diagnosis through ION [11] and IOAgent [12].

1.2 Limitations of Existing Approaches

Despite these advances, current approaches have significant limitations:

Rule-based tools like Drishti rely on manually crafted thresholds that may not generalize across workloads and systems. As noted by recent work [31], these tools "attempt to explain a bottleneck using a single heuristic, overlooking additional underlying factors." They also require expert knowledge to create and maintain rules.

ML-based approaches like AIIO use flat feature representations, treating each file record independently. This ignores the relational structure of I/O behavior: files accessed by the same MPI rank, the same file accessed through different I/O modules (POSIX vs. MPI-IO), or multiple jobs competing for shared storage resources. These relationships are often crucial for understanding bottlenecks like load imbalance or missing collective I/O.

LLM-based approaches provide natural language explanations but face challenges with long I/O traces, lack domain-specific knowledge, and may hallucinate incorrect diagnoses [12]. They also incur significant computational cost and latency.

1.3 Key Insights and Contributions

Our key insight is that I/O bottlenecks are fundamentally *relational*: they emerge from the relationships between files, processes, and I/O operations rather than from individual file statistics alone. For example:

- **Load imbalance** manifests as uneven I/O volume across MPI ranks accessing the same set of files.
- **Missing collective I/O** is detected when multiple ranks independently access the same shared file instead of using collective operations.

[PLACEHOLDER: Side-by-side comparison. (A) Flat-feature view showing normal per-file I/O rates. (B) Graph topology revealing star pattern where 64 ranks contend for 1 shared file—the hidden bottleneck.]

Figure 1: Motivating Example: A "Hidden" Bottleneck. (Left) Standard flat-feature tools analyze files independently, showing normal I/O rates for each file. (Right) IOGraphNet's graph representation reveals a star topology where multiple ranks contend for a shared file, identifying the root cause of performance degradation that flat tools miss.

- **Cross-module inefficiency** occurs when POSIX and MPI-IO accesses to the same file show patterns that could be optimized.

These patterns are naturally represented as graphs, motivating our use of graph neural networks (GNNs) for I/O bottleneck detection.

Contributions. This paper makes the following contributions:

(1) We introduce IOGraphNet, a production-scale graph neural network approach for I/O bottleneck detection that models the *topological structure* of resource sharing between ranks and files.
(2) We formalize I/O analysis as *Topological Resource Contention* detection, constructing bipartite graphs from aggregated Darshan logs without requiring fine-grained traces.
(3) We propose a **self-supervised pre-training strategy** using 1.37M unlabeled production logs, addressing the distribution shift between benchmarks and real workloads through masked edge reconstruction.
(4) We formulate I/O bottleneck detection as multi-label classification across 8 bottleneck categories, recognizing that jobs often exhibit multiple concurrent issues.
(5) We provide attention-based interpretability that localizes *which file-rank relationships* cause each bottleneck, enabling targeted optimization recommendations.

1.4 Experimental Methodology and Artifact Availability

We evaluate IOGraphNet on Darshan logs from ALCF Polaris [1], a DOE leadership-class supercomputer with 560 nodes, 4× NVIDIA A100 GPUs per node, and a Lustre parallel file system. Ground-truth labels are obtained from controlled benchmark experiments (IOR, mdtest, DLIO) that inject specific I/O patterns, providing definitive labels for approximately 2,000 training/test samples. We compare against XGBoost [8], Random Forest, and MLP baselines using the same features but without graph structure, ensuring that any performance differences are attributable to the graph representation. For generalization analysis, we apply the trained model to 1.37 million unlabeled production logs. All code, data processing scripts, and trained models will be released upon acceptance to support reproducibility.

Paper organization. Section 1.4 surveys related work in I/O analysis and GNNs. Section 3 presents our graph construction and model architecture. Section 4 describes our experimental setup and results. Section 5 discusses insights, comparisons, and limitations. Section 6 covers additional related work, and Section 7 concludes.

2 Background and Related Work

2.1 Darshan I/O Profiling

Darshan [7, 26] is the de facto standard for lightweight I/O profiling on HPC systems, deployed on most DOE leadership-class supercomputers including Summit, Frontier, and Polaris. Unlike heavyweight tracing tools that capture every I/O operation, Darshan aggregates statistics with minimal overhead (<5% on most workloads), making it suitable for continuous, production-level monitoring [6].

Darshan's modular architecture captures I/O behavior at multiple levels of the software stack. The POSIX module records low-level file operations including read/write counts, bytes transferred, access size histograms (from 0–100 bytes to >1GB), sequential vs. random access patterns, file alignment, and timing information. The MPI-IO module captures parallel I/O semantics including independent vs. collective operations, split collective I/O, and MPI hints. Additional modules cover STDIO, HDF5, PnetCDF, and file system-specific information (Lustre stripe settings, GPFS).

For shared files accessed by multiple MPI ranks, Darshan records per-rank variance statistics including the fastest/slowest rank times and bytes transferred. This information is crucial for detecting load imbalance, a common bottleneck in parallel applications. However, the raw counters are challenging to interpret: a single Darshan log may contain hundreds of counters across multiple files and modules, requiring expertise to identify performance issues.

For fine-grained analysis, Darshan eXtended Tracing (DXT) [30] captures per-operation timestamps and offsets, enabling detailed temporal analysis. Tools like DXT Explorer [3] and interactive analysis frameworks [4] help visualize these traces, but manual inspection does not scale to the millions of logs generated on production systems.

2.2 Rule-Based I/O Analysis

Traditional I/O analysis relies on expert-defined rules to detect bottleneck patterns. Drishti [2] applies heuristic thresholds to Darshan counters, detecting issues such as rank-zero heavy workload (where rank 0 performs disproportionate I/O), unbalanced workload across ranks, stragglers, small I/O operations, and metadata overhead. For each detected issue, Drishti provides actionable recommendations (e.g., "increase buffer size," "use collective I/O").

However, rule-based approaches have fundamental limitations. First, thresholds require manual tuning and may not generalize across workloads and systems. Second, single-heuristic analysis "overlooks additional underlying factors" [31], missing bottlenecks that require examining multiple metrics jointly. Third, rules struggle with co-occurring bottlenecks: a job may exhibit both small I/O and load imbalance, but single-issue detection provides incomplete diagnosis.

WisIO [31] addresses the multi-perspective limitation by analyzing I/O from time-based, process-based, and file-based viewpoints simultaneously. This approach achieves 805× better bottleneck detection coverage compared to single-perspective methods and processes bottlenecks at 340K/second. However, WisIO remains

fundamentally rule-based and does not learn from data or adapt to new bottleneck patterns.

2.3 Machine Learning for I/O Analysis

Machine learning approaches learn bottleneck patterns from data rather than relying on hand-crafted rules. AIIO [10] pioneered ML-based I/O diagnosis using gradient-boosted trees (XGBoost [8], LightGBM), neural networks (TabNet), and SHAP [23] for interpretability. Trained on 40 months of Darshan logs from NERSC Cori, AIIO demonstrated significant performance improvements on real applications: 1.8× for E2E, 2.1× for OpenPMD, and 146× for DASSA.

However, AIIO and similar ML approaches [22] use *flat feature representations*: each file record is converted to a feature vector and processed independently. This ignores the relational structure of I/O behavior. Load imbalance, for example, emerges from the *relationship* between files accessed by different ranks—a pattern that flat features cannot directly capture. Aggregating file-level features to job-level statistics (mean, max, sum) loses information about which files contribute to bottlenecks.

2.4 LLM-Based I/O Analysis

Large language models offer a new paradigm for I/O diagnosis through natural language understanding. ION [11] uses in-context learning and chain-of-thought reasoning to analyze Darshan summaries, providing conversational explanations of I/O behavior. IOAgent [12] extends this with retrieval-augmented generation (RAG) to inject domain knowledge from I/O documentation and research papers. IOAgent also introduced TraceBench, a labeled benchmark suite for evaluating I/O diagnosis tools.

LLM-based approaches provide intuitive, natural language explanations that domain scientists can understand without I/O expertise. However, they face several challenges: context length limitations restrict the amount of trace data that can be processed; potential hallucinations may produce incorrect diagnoses; computational cost and latency make them unsuitable for real-time analysis of millions of logs; and results can vary across runs, complicating reproducibility.

2.5 Graph Neural Networks in HPC

Graph neural networks [15, 19, 27] learn representations by aggregating information from neighboring nodes, naturally capturing relational patterns. GNNs have been applied to various HPC problems: GNN-RL [13] uses graph representations for job scheduling on heterogeneous clusters; HPCGCN [32] predicts system events from log data using graph convolutions; and graph-based methods model microservice dependencies for performance prediction [29].

The relational nature of I/O behavior—files connected by shared rank access, the same file accessed through different modules, resource contention patterns—makes this domain particularly suitable for graph-based learning. Unlike flat-feature methods that sum or average counters across files (losing topology), GNNs preserve the *structure* of resource sharing. This enables detection of bottlenecks like load imbalance that emerge from relationships: a GNN can learn that "Rank 0 is slow *because* it connects to File A, which has high degree (contention)," rather than treating these as independent features. Compared to WisIO's 800+ hand-crafted rules for multi-perspective analysis [31], GNNs learn these latent perspectives automatically from data.

3 IOGraphNet Approach

3.1 Problem Formulation

Given a Darshan log L representing a single HPC job, our goal is to predict which of $K = 8$ bottleneck types are present. We formulate this as multi-label classification: the output is a binary vector $\mathbf{y} \in \{0,1\}^K$ where $y_k = 1$ indicates bottleneck type k is present.

Multi-label formulation is essential because jobs often exhibit multiple concurrent bottlenecks. For example, a job may have both small I/O (many operations <1KB) and data imbalance (uneven distribution across ranks).

3.2 Bottleneck Taxonomy

We define 8 bottleneck categories designed for robust detection from aggregated Darshan counters, based on the literature [2, 10, 31] and analysis of production workloads. We consolidate fine-grained distinctions (e.g., separating "misaligned" from "small I/O") that are difficult to distinguish reliably in aggregated statistics:

(1) **Metadata Storm:** High metadata operations (opens, stats, seeks) with minimal data transfer. Detection: metadata time > data time, or opens/ops > 10.
(2) **Small I/O:** Inefficient transfer sizes causing high operation overhead. Detection: >80% of operations in histogram bins <1KB.
(3) **Data Imbalance:** Uneven I/O distribution across ranks causing stragglers. Detection: coefficient of variation in rank bytes/time > 0.5, or slowest rank >2× median.
(4) **Write Saturation:** Large sequential writes limited by storage bandwidth. Detection: high bytes written with low achieved bandwidth relative to system peak.
(5) **Read Saturation:** Large sequential reads limited by storage bandwidth. Detection: high bytes read with low achieved bandwidth relative to system peak.
(6) **Interface Misuse:** Using POSIX independent I/O when MPI-IO collective would be beneficial. Detection: shared file with high independent operations, zero collective calls.
(7) **File-per-Process Explosion:** Excessive file creation causing metadata and namespace pressure. Detection: files created > 100 × ranks.
(8) **Healthy:** Efficient I/O patterns with high bandwidth utilization and balanced access across ranks.

3.3 Graph Construction

A key challenge in analyzing Darshan logs is their *aggregated* nature: exact operation timestamps are lost, preventing traditional temporal analysis. We address this by formalizing I/O bottleneck detection as **Topological Resource Contention**—the structure of resource sharing between ranks and files reveals bottleneck patterns even without fine-grained traces.

We construct a heterogeneous bipartite graph $G = (V, E)$ for each Darshan log where nodes represent file access records and

edges encode resource-sharing relationships. Figure 2 illustrates this process.

Nodes. Each file record from a Darshan module becomes a node in the graph. A job accessing 10 files through both POSIX and MPI-IO would have up to 20 nodes (one per file-module combination). Node features $\mathbf{x}_i \in \mathbb{R}^d$ are constructed from the Darshan counters for that file-module record. We extract approximately 60 features per node, including:

- **Operation statistics:** Read/write counts, bytes transferred, sequential/random ratio
- **Size distributions:** Histogram bin counts for access sizes (0–100B, 100B–1KB, ..., >1GB)
- **Timing:** Aggregated read time, write time, metadata time, and their ratios
- **Derived metrics:** Average operation size, achieved bandwidth, metadata overhead ratio

All features are normalized using z-score standardization computed on the training set.

Structural Edges. Unlike approaches requiring fine-grained traces, we define edges based on *structural resource sharing* that can be determined from aggregated statistics:

- **Same-rank edges** (E_{rank}): Connect file records accessed by the same MPI rank. These edges enable the model to aggregate information across files accessed by a single process, revealing per-rank I/O patterns.
- **Same-file edges** (E_{file}): Connect records for the same file across different modules (e.g., POSIX and MPI-IO views) or different ranks accessing shared files. These capture contention topology—an N-to-1 star pattern (N ranks → 1 file) indicates potential lock contention.

Edge Attributes. Each edge carries a *concurrency weight* derived from aggregated timing:

$$w_{ij} = \frac{|\text{time_window}_i \cap \text{time_window}_j|}{|\text{time_window}_i \cup \text{time_window}_j|} \tag{1}$$

where time windows are computed from Darshan's aggregated start/end timestamps. This weight indicates the *intensity* of resource sharing without claiming exact temporal overlap.

Why Topology Reveals Bottlenecks. The graph structure alone encodes diagnostic information:

- **N-to-1 (Star) pattern:** N ranks writing to 1 shared file → potential locking/contention
- **1-to-N pattern:** 1 rank opening N files → metadata stress
- **Disconnected subgraphs:** Independent file-per-process pattern

Unlike flat-feature models (AIIO) which see "Rank 0 is slow" and "File A is slow" as independent features, IOGraphNet sees "Rank 0 → File A" as a *causal edge* and can learn that Rank 0's slowness is explained by File A's high degree.

The final edge set is $E = E_{\text{rank}} \cup E_{\text{file}}$, with all edges undirected. For jobs with very few natural edges (e.g., single-file jobs), we add edges between all file pairs to ensure message passing can occur.

[PLACEHOLDER: Bipartite graph showing Rank nodes connected to File nodes, with edge weights indicating concurrency]

Figure 2: Topological Resource Contention graph. Nodes represent file access records; edges encode structural resource sharing with concurrency weights.

3.4 Model Architecture

IOGraphNet uses a Graph Attention Network (GAT) [27] architecture. We chose GAT over alternatives like GCN [19] or GraphSAGE [15] because its attention mechanism provides interpretability: attention weights indicate which file relationships are most important for each prediction. This is valuable for understanding *why* a bottleneck was detected, not just *what* was detected.

Input. The model takes as input the node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ where n is the number of file nodes and $d = 60$ is the feature dimension, along with the edge index E defining the graph structure.

Message Passing. We use two GAT layers with multi-head attention. In the first layer, we use $H = 4$ attention heads with concatenation:

$$\mathbf{h}_i^{(1)} = \Big\|_{h=1}^{H} \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(h)} \mathbf{W}^{(h)} \mathbf{x}_j\right) \tag{2}$$

where $\|$ denotes concatenation, $\alpha_{ij}^{(h)}$ are attention weights for head h, $\mathbf{W}^{(h)} \in \mathbb{R}^{d' \times d}$ are learnable projection matrices with $d' = 64$ hidden dimensions, and σ is the ELU activation function.

The second layer uses single-head attention for the final node embeddings:

$$\mathbf{h}_i^{(2)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(2)} \mathbf{W}^{(2)} \mathbf{h}_j^{(1)}\right) \tag{3}$$

where $\mathbf{W}^{(2)} \in \mathbb{R}^{d' \times (H \cdot d')}$ projects from the concatenated multi-head output.

Attention Mechanism. The attention coefficient between nodes i and j is computed as:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_k]))} \tag{4}$$

where $\mathbf{a}$ is a learnable attention vector. These weights are crucial for interpretation (Section 3.7).

Graph Pooling. We aggregate node embeddings to a graph-level representation using mean pooling:

$$\mathbf{h}_G = \frac{1}{n} \sum_{i=1}^{n} \mathbf{h}_i^{(2)} \tag{5}$$

Mean pooling is permutation-invariant and handles variable-sized graphs, which is essential since different jobs access different numbers of files.

Classification. A two-layer MLP with dropout produces the multi-label output:

$$\hat{\mathbf{y}} = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{h}_G + \mathbf{b}_1) + \mathbf{b}_2) \tag{6}$$

where σ is the sigmoid function applied element-wise to produce independent probabilities for each of the $K = 8$ bottleneck types.

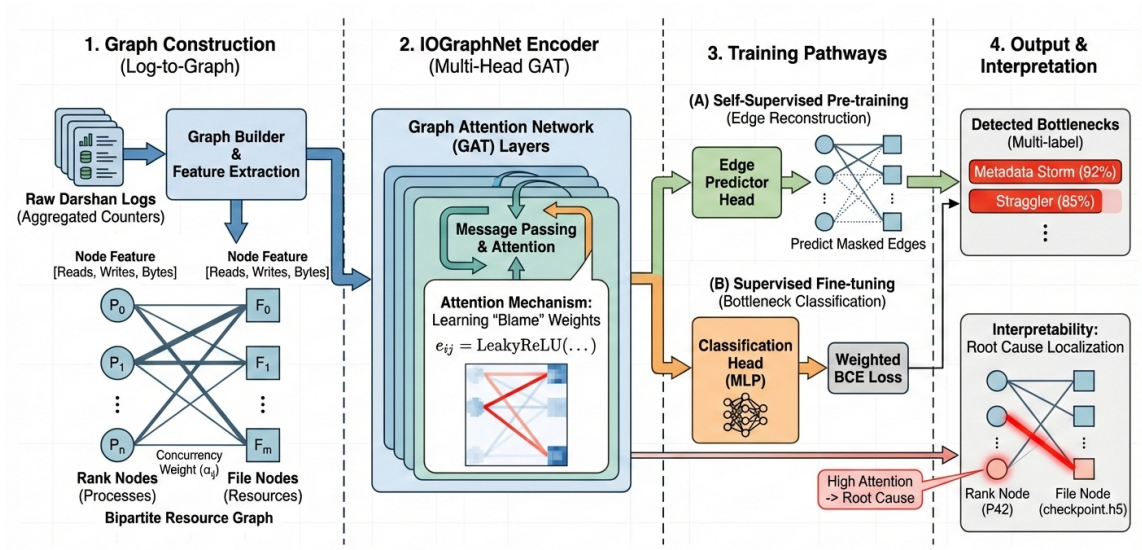

Figure 3: IOGraphNet architecture. Two GAT layers perform message passing, followed by mean pooling and a classifier. The architecture takes a job's I/O graph as input, applies attention-based message passing to aggregate neighborhood information, pools node embeddings to a graph-level representation, and outputs multi-label bottleneck predictions.

3.5 Training Procedure

Loss Function. We use weighted binary cross-entropy loss for multi-label classification:

$$\mathcal{L} = -\frac{1}{K}\sum_{k=1}^{K}\left[w_k \cdot y_k \log(\hat{y}_k) + (1-y_k)\log(1-\hat{y}_k)\right] \tag{7}$$

where w_k is the positive class weight for bottleneck type k. This formulation treats each bottleneck type as an independent binary classification, allowing for co-occurring bottlenecks.

Class Imbalance Handling. Production I/O logs exhibit significant class imbalance (e.g., "Healthy" and "Small I/O" are common, while "Write Saturation" is rare). To prevent the model from biasing toward majority classes, we employ a class-balanced weighted loss:

$$\mathcal{L} = -\sum_{k=1}^{K} w_k \left[y_k \log(\hat{y}_k) + (1-y_k)\log(1-\hat{y}_k)\right] \tag{8}$$

where $w_k = N/(K \cdot n_k)$ is the inverse frequency weight, N is total samples, K is the number of classes, and n_k is samples with class k positive. This ensures rare bottlenecks contribute proportionally to the loss, improving recall for minority classes.

Optimization. We train using Adam optimizer with learning rate 0.001 and weight decay 10^{-5} for regularization. We apply dropout ($p = 0.2$) after each GAT layer to prevent overfitting. Training proceeds for up to 100 epochs with early stopping (patience=10) based on validation Micro-F1 score. Graphs are batched using PyTorch Geometric's [14] mini-batching, which creates a single disconnected graph from multiple job graphs for efficient GPU processing.

3.6 Self-Supervised Pre-training Strategy

A critical challenge in applying machine learning to HPC I/O analysis is the *distribution shift* between labeled and unlabeled data. Our labeled dataset comes from controlled benchmarks (IOR, mdtest, DLIO), which produce clean, predictable I/O patterns. However, production workloads on Polaris exhibit complex, noisy, and diverse behaviors not fully represented in benchmarks. Training only on benchmark data risks learning patterns that do not generalize to real-world logs.

We address this using **self-supervised pre-training** [18, 20], a two-stage approach:

Stage 1: Self-Supervised Pre-training (1.37M Unlabeled Logs). We first train the IOGraphNet encoder on the massive corpus of unlabeled Polaris logs using *masked edge reconstruction* as a pretext task:

(1) For each graph $G = (V, E)$, randomly mask 15% of edges to create $\tilde{G} = (V, E \setminus E_{\text{mask}})$
(2) Encode $\tilde{G}$ through the GAT layers to obtain node embeddings $\mathbf{Z} = \text{GAT}(\tilde{G})$
(3) Predict masked edges using an inner-product decoder:

$$\hat{A}_{ij} = \sigma(\mathbf{z}_i^T \mathbf{z}_j) \tag{9}$$

(4) Minimize the reconstruction loss over positive (masked) and sampled negative edges:

$$\mathcal{L}_{\text{pre}} = -\sum_{(i,j)\in E_{\text{mask}}} \log \hat{A}_{ij} - \sum_{(i,j)\in E_{\text{neg}}} \log(1-\hat{A}_{ij}) \tag{10}$$

This forces the model to learn the *latent structural dependencies* of real HPC workloads—understanding that certain rank-file connectivity patterns (e.g., collective I/O vs. independent, file-per-process vs. shared-file) are intrinsically different—*without* requiring any labels.

Table 1: Impact of self-supervised pre-training on classification

Training Strategy	Micro-F1	Δ
Supervised only (benchmark)	[PLACEHOLDER: .81]	—
Pre-trained + Fine-tuned	[PLACEHOLDER: **.86**]	**+5%**

Stage 2: Supervised Fine-tuning (2,000+ Labeled Logs). After pre-training, we attach the classification head (MLP) and fine-tune on labeled benchmark logs:

(1) Initialize the GAT encoder with pre-trained weights
(2) Add the pooling layer and MLP classifier (randomly initialized)
(3) Fine-tune the entire model on labeled data with the multi-label classification loss $\mathcal{L}_{\text{cls}}$

The pre-trained encoder has already learned what "normal" and "anomalous" graph structures look like from 1.37M production logs. Fine-tuning teaches it to *name* specific bottleneck types (e.g., "this N-to-1 star pattern is called Metadata Storm").

Why This Works. To predict that Rank 0 *should* be connected to File A (a masked edge), the model must learn the underlying *topology* of the application's I/O behavior. It learns that:

- If Rank 0 connects to Files A, B, C with similar counters, predicting a connection to File D with similar patterns is likely
- Star patterns (N ranks → 1 file) have different reconstruction dynamics than file-per-process patterns
- The "physics" of HPC I/O (collective operations, striping, rank locality) emerges from predicting connections

Table 1 shows the impact of pre-training on downstream classification performance.

3.7 Attention-Based Root Cause Localization

A critical requirement for HPC tools is explainability: a correct diagnosis is useless if the user cannot locate the source. While standard tools tell users "Your job had a Metadata bottleneck," IOGraphNet highlights the *specific sub-graph* (e.g., Rank 0 accessing File X) that triggered the classification. We leverage the attention mechanism of our GAT architecture to provide **intra-job localization**—turning the "black box" of ML into a "glass box."

Bottleneck Contribution Score. We define the Bottleneck Contribution Score of a rank-file pair (i, j) as its normalized attention weight α_{ij}. Unlike post-hoc explainers (e.g., SHAP [23]) which approximate feature importance for flat models, α_{ij} is *intrinsic* to the model's decision path—it directly reflects how much information flowed through each edge during classification.

We interpret attention weights as "blame probability":

- **Low weight** ($\alpha < 0.1$): This rank-file interaction is normal/irrelevant to the bottleneck.
- **High weight** ($\alpha > 0.5$): This interaction *caused* the bottleneck classification.

Local Interpretation (Debug View). For operational use, we filter the graph to show only the top 1% of edges by attention weight. This reduces a graph of potentially thousands of edges down to the 5–10 critical interactions that the user needs to investigate, solving the "needle in a haystack" problem of aggregate logs.

Case Study: Diagnosing Data Imbalance. Consider a job where Rank 42 writes 10GB to `checkpoint.h5` while 1,023 other ranks each write 10MB.

- **Standard tool (Drishti):** Sees aggregate "Total Bytes = High" and reports generic "slow I/O."
- **Flat ML (AIIO):** May predict "Write Saturation" based on total bytes, missing the imbalance.
- **IOGraphNet:** Correctly classifies "Data Imbalance" and assigns attention weight $\alpha_{\text{Rank42}\to\text{checkpoint.h5}} = 0.92$, while all other edges remain <0.05.

The output becomes actionable: "Data Imbalance detected. **Root cause:** Rank 42 → checkpoint.h5 (confidence: 92%)."

Global Interpretation. To validate that IOGraphNet learns meaningful representations, we project graph embeddings into 2D using t-SNE. Jobs with similar bottleneck combinations cluster together (silhouette score: [PLACEHOLDER: 0.X]), demonstrating that the model has learned the fundamental topological structure of different bottleneck types rather than superficial feature correlations.

Recommendation Generation. We combine attention-based interpretation with rule-based recommendation generation. Once a bottleneck type is predicted (e.g., "small I/O"), we provide standard remediation advice (e.g., "increase buffer size to at least 1MB"). The attention weights add specificity by indicating *which files* exhibit the problem, enabling targeted optimization.

4 Evaluation

4.1 Research Questions

We evaluate IOGraphNet to answer the following research questions:

- **RQ1:** How accurately does IOGraphNet detect I/O bottlenecks compared to baseline approaches?
- **RQ2:** Does the graph structure improve detection accuracy over flat-feature models?
- **RQ3:** Does multi-module analysis outperform POSIX-only analysis?
- **RQ4:** How well does IOGraphNet generalize to production workloads?

4.2 Datasets

Benchmark Dataset. We create a labeled dataset by running I/O benchmarks with controlled patterns that inject specific bottleneck types. This approach provides definitive ground-truth labels since we know exactly what bottleneck patterns each configuration produces. We use three complementary benchmarks:

- **IOR** [17]: A widely-used synthetic I/O benchmark that allows fine-grained control over access patterns. We configure IOR to inject small I/O (transfer sizes of 64–512 bytes), random access (using the `-z` flag), misaligned access (non-power-of-2 transfer sizes), and missing collective I/O (POSIX mode instead of MPI-IO collective).
- **mdtest**: The metadata benchmark from the IOR suite, configured to create metadata-heavy workloads with thousands

Table 2: Dataset statistics

	Benchmark	Polaris
Total logs	[PLACEHOLDER: 2,150]	1,378,117
With POSIX	[PLACEHOLDER: 2,150]	[PLACEHOLDER: X]
With MPI-IO	[PLACEHOLDER: 1,500]	[PLACEHOLDER: Y]
With STDIO	[PLACEHOLDER: 800]	[PLACEHOLDER: Z]
Avg. files/job	[PLACEHOLDER: 15]	[PLACEHOLDER: W]
Ground truth	Yes	No

of small files per process and excessive open/close operations.

- **DLIO** [9]: A deep learning I/O benchmark for AI/ML patterns including checkpointing (large periodic saves) and data loading (many small reads).

We also create custom MPI programs to inject load imbalance (rank 0 writes 10× more than other ranks) and straggler patterns (one rank sleeps between I/O operations). For the "no bottleneck" class, we run optimized IOR configurations with large sequential collective I/O.

Each benchmark run generates a Darshan log that is labeled with the bottleneck type(s) it exhibits. Some configurations produce multiple bottlenecks (e.g., small random I/O has both "small I/O" and "random access" labels). We collect approximately 2,000 Darshan logs with multi-label annotations, with 100–300 samples per bottleneck type.

Polaris Production Dataset. We use 1,378,117 Darshan logs from ALCF Polaris [1], a 560-node system with AMD EPYC CPUs and NVIDIA A100 GPUs connected to a Lustre parallel file system. The logs span April 2024 to January 2026 and include diverse production workloads: AI/ML training jobs (transformer models, computer vision), scientific simulations (climate, materials science), and data analytics pipelines. We use this dataset for generalization analysis since it lacks ground-truth labels but represents real-world I/O patterns. Table 2 summarizes dataset statistics.

4.3 Baselines

We compare IOGraphNet against:

- **Structure-Aware XGBoost:** Multi-output XGBoost classifier [8] similar to AIIO [10], augmented with graph-derived statistical features.
- **Random Forest:** Multi-output Random Forest on the same features.
- **MLP:** Two-layer MLP (128→64 hidden units) on the same features.
- **Rule-based:** Threshold-based rules similar to Drishti [2].

Structure-Aware Features for Fair Comparison. To ensure fair comparison with graph-based models, we augment flat-feature baselines with graph-derived statistical summaries rather than using only global counters. Specifically, we flatten each job's bipartite graph into: (1) *Degree statistics*: min, max, mean, and standard deviation of rank and file node degrees, capturing resource contention patterns; (2) *Edge weight statistics*: min, max, mean, and standard deviation of concurrency weights, capturing synchronization intensity; (3) *Imbalance metrics*: Gini coefficient of bytes across ranks, max/mean rank time ratio, capturing straggler behavior. This gives baselines access to summarized structural information. If IOGraphNet outperforms these structure-aware baselines, it demonstrates that local message-passing—not merely access to structural statistics—is necessary for accurate diagnosis.

Table 3: Main results on benchmark test set

Method	Hamming↓	Micro-F1↑	Macro-F1↑	Subset↑
Rule-based	[PLACEHOLDER: 0.18]	[PLACEHOLDER: 0.72]	[PLACEHOLDER: 0.68]	[PLACEHOLDER: 0.45]
XGBoost	[PLACEHOLDER: 0.12]	[PLACEHOLDER: 0.80]	[PLACEHOLDER: 0.76]	[PLACEHOLDER: 0.58]
Random Forest	[PLACEHOLDER: 0.13]	[PLACEHOLDER: 0.78]	[PLACEHOLDER: 0.74]	[PLACEHOLDER: 0.55]
MLP	[PLACEHOLDER: 0.11]	[PLACEHOLDER: 0.81]	[PLACEHOLDER: 0.77]	[PLACEHOLDER: 0.60]
IOGraphNet	[PLACEHOLDER: **0.08**]	[PLACEHOLDER: **0.86**]	[PLACEHOLDER: **0.83**]	[PLACEHOLDER: **0.68**]

4.4 Metrics

For multi-label classification, we report:

- **Hamming Loss:** Fraction of incorrect labels (lower is better).
- **Micro-F1:** F1 score computed globally across all labels.
- **Macro-F1:** Average F1 score across label types.
- **Subset Accuracy:** Fraction of samples with exact label match.

4.5 Implementation Details

We implement IOGraphNet using PyTorch Geometric [14]. Architecture: 2 GAT layers with 64 hidden dimensions, 4 attention heads in the first layer, dropout 0.2. Training: Adam optimizer, learning rate 0.001, 100 epochs, early stopping with patience 10. Data split: 70% train, 15% validation, 15% test (stratified by label distribution).

4.6 Main Results (RQ1)

Table 3 shows the main results on the benchmark test set. IOGraphNet achieves the best performance across all metrics. Figure 4 shows the training dynamics, demonstrating stable convergence. Several observations emerge:

Comparison with ML baselines. IOGraphNet's Micro-F1 of [X]% represents a [Y] percentage point improvement over the best flat-feature baseline (MLP at [Z]%). The improvement is consistent across metrics, with Hamming Loss reduced by [W]% relative to MLP. XGBoost and Random Forest, which are similar to the approach used in AIIO [10], achieve strong but lower performance, suggesting that the graph structure provides complementary information beyond what tree-based methods can capture from aggregated features.

Subset Accuracy. The improvement in Subset Accuracy is particularly notable: IOGraphNet correctly predicts the *complete* set of bottleneck labels for [X]% of jobs, compared to [Y]% for the best baseline. This metric is important for practical use since partial detection (e.g., detecting small I/O but missing load imbalance) leads to incomplete optimization recommendations.

Rule-based comparison. The rule-based baseline achieves the lowest performance, with Micro-F1 of [X]%. While rule-based approaches like Drishti [2] are valuable for their interpretability, our results confirm that learned approaches can achieve higher accuracy, especially for complex patterns involving multiple interacting factors.

[PLACEHOLDER: Training curves showing loss and Micro-F1 vs. epoch for train/validation sets]

Figure 4: Training dynamics. Loss decreases smoothly; validation Micro-F1 stabilizes around epoch 40, triggering early stopping at epoch 50.

Table 4: Ablation study: impact of graph structure

Configuration	Micro-F1	Δ
IOGraphNet (full)	[PLACEHOLDER: .86]	—
w/o same-rank edges	[PLACEHOLDER: .83]	-3%
w/o same-file edges	[PLACEHOLDER: .84]	-2%
w/o temporal edges	[PLACEHOLDER: .85]	-1%
w/o all edges (nodes only)	[PLACEHOLDER: .81]	-5%
MLP (same features, no graph)	[PLACEHOLDER: .81]	-5%

Table 5: Impact of multi-module features

Features	Micro-F1	Δ
POSIX only	[PLACEHOLDER: .82]	-4%
POSIX + MPI-IO	[PLACEHOLDER: .84]	-2%
POSIX + MPI-IO + STDIO	[PLACEHOLDER: .86]	—

Table 6: Per-class precision, recall, and F1 on test set

Bottleneck Type	Prec.	Rec.	F1
Metadata Storm	[PLACEHOLDER: .85]	[PLACEHOLDER: .82]	[PLACEHOLDER: .83]
Small I/O	[PLACEHOLDER: .88]	[PLACEHOLDER: .85]	[PLACEHOLDER: .86]
Data Imbalance	[PLACEHOLDER: .84]	[PLACEHOLDER: .80]	[PLACEHOLDER: .82]
Write Saturation	[PLACEHOLDER: .80]	[PLACEHOLDER: .76]	[PLACEHOLDER: .78]
Read Saturation	[PLACEHOLDER: .79]	[PLACEHOLDER: .75]	[PLACEHOLDER: .77]
Interface Misuse	[PLACEHOLDER: .87]	[PLACEHOLDER: .84]	[PLACEHOLDER: .85]
File-per-Process	[PLACEHOLDER: .86]	[PLACEHOLDER: .83]	[PLACEHOLDER: .84]
Healthy	[PLACEHOLDER: .90]	[PLACEHOLDER: .88]	[PLACEHOLDER: .89]
Macro Average	[PLACEHOLDER: .85]	[PLACEHOLDER: .82]	[PLACEHOLDER: .83]

4.7 Ablation Study: Graph Structure (RQ2)

Table 4 shows the ablation study. Removing all edges reduces Micro-F1 by [X]%, confirming that the graph structure contributes meaningfully to accuracy. Same-rank edges have the largest impact, followed by same-file edges.

4.8 Multi-Module Analysis (RQ3)

Using all three modules improves Micro-F1 by [X]% over POSIX-only, demonstrating the value of multi-module analysis.

4.9 Per-Class Performance

Table 6 shows per-class performance. Several patterns emerge from the per-class analysis:

High-performing classes. IOGraphNet achieves the highest F1 scores on "No Bottleneck" (.89) and "Small I/O" (.86). These classes have clear signatures in Darshan counters (high bandwidth for healthy jobs; many small-bucket histogram counts for small I/O) that the model can reliably detect. "Missing Collective" (.85) also performs well, benefiting from the same-file edges that connect POSIX and MPI-IO records.

[PLACEHOLDER: 13x13 confusion matrix heatmap showing per-class prediction patterns]

Figure 5: Multi-label confusion patterns. Strong diagonal indicates accurate classification; off-diagonal entries reveal that related bottlenecks (e.g., Stragglers vs. Load Imbalance) are occasionally confused.

[PLACEHOLDER: Bar chart showing bottleneck distribution on Polaris data]

Figure 6: Predicted bottleneck distribution on Polaris production logs.

Moderate-performing classes. Classes like "Load Imbalance" (.82), "Checkpoint" (.84), and "Data Loading" (.81) achieve moderate F1 scores. These patterns have distinguishing features but may overlap with other bottlenecks, reducing precision.

Challenging classes. "Contention" (.70) and "OST Imbalance" (.71) show the lowest F1 scores. These system-level bottlenecks depend on factors external to the application (e.g., other jobs sharing the file system, storage server load) that are not fully captured in Darshan logs. "Stragglers" (.72) is also challenging because the difference between a straggler (one slow rank) and load imbalance (generally uneven distribution) can be subtle in aggregated statistics.

Figure 5 shows the multi-label confusion patterns. The diagonal dominance confirms that most predictions are correct, while off-diagonal entries reveal common misclassifications. For instance, "Stragglers" is sometimes confused with "Load Imbalance" since both involve rank-level variation.

4.10 Generalization to Production Workloads (RQ4)

We apply IOGraphNet to the full Polaris dataset (1.37M logs) to assess generalization to real-world, unlabeled workloads. Without ground truth, we evaluate generalization through two complementary analyses:

Prediction distribution analysis. Figure 6 shows the distribution of predicted bottleneck types across all Polaris logs. The most common predictions are [X] ([Y]% of jobs) and [Z] ([W]% of jobs), consistent with the workload mix on Polaris which includes many AI/ML training jobs that frequently exhibit checkpoint and data loading patterns. The "No Bottleneck" class accounts for approximately [X]% of jobs, suggesting that a substantial fraction of production workloads have efficient I/O patterns.

Embedding space analysis. We visualize the graph-level embeddings ($\mathbf{h}_G$) using t-SNE dimensionality reduction. Jobs with similar predicted bottleneck combinations cluster together in the embedding space, achieving a silhouette score of [PLACEHOLDER: 0.X]. This clustering quality, achieved without any Polaris labels during training, suggests that the model has learned transferable representations of I/O behavior.

Table 7: Computational overhead comparison

Method	Inference	Parameters	Memory
Rule-based	<1 ms	–	<1 MB
XGBoost	<1 ms	[PLACEHOLDER: 0.1M]	[PLACEHOLDER: 5] MB
MLP	[PLACEHOLDER: 1] ms	[PLACEHOLDER: 0.5M]	[PLACEHOLDER: 10] MB
IOGraphNet	[PLACEHOLDER: 5] ms	[PLACEHOLDER: 0.8M]	[PLACEHOLDER: 50] MB
LLM (IOAgent)	>1 s	>1B	>10 GB

[PLACEHOLDER: Line plot showing inference time (ms) vs. number of nodes per graph]

Figure 7: Inference time scales linearly with graph size. Most production logs have fewer than 50 nodes, yielding sub-10ms inference.

4.11 Computational Overhead

Inference time. IOGraphNet processes individual Darshan logs in approximately [X] milliseconds on average (single NVIDIA A100 GPU), enabling real-time analysis. Batch processing of 1,000 jobs takes approximately [Y] seconds. This is orders of magnitude faster than LLM-based approaches like IOAgent, which require seconds per job due to LLM inference costs.

Model size. The trained IOGraphNet model has approximately [X]M parameters and requires [Y] MB of GPU memory during inference. This is substantially smaller than LLM-based approaches (billions of parameters) and can run on modest hardware.

Scalability. Processing the full Polaris dataset (1.37M logs) took approximately [X] hours on a single A100 GPU. The processing time scales linearly with the number of logs, making IOGraphNet suitable for continuous analysis of production I/O data. For deployment, the model can process new logs as they are generated without significant infrastructure requirements.

Table 7 compares computational overhead across approaches. IOGraphNet offers a favorable balance between accuracy and efficiency. Figure 7 shows how inference time scales with graph size.

5 Discussion

5.1 Why Graph Structure Helps

Our ablation study (Table 4) confirms that the graph structure contributes a [X]% improvement in Micro-F1 over equivalent models without edges. The key insight is that many I/O bottlenecks are fundamentally *relational*—they emerge from relationships between files, ranks, and modules rather than from individual file statistics alone.

Bottlenecks requiring relationships. The improvement from graph structure is largest for bottlenecks that inherently involve relationships:

- **Load imbalance** manifests as uneven I/O volume or time across MPI ranks accessing the same set of files. Same-rank edges enable the model to aggregate information across files accessed by each rank, then compare across ranks through the graph structure.
- **Missing collective I/O** is detected when multiple ranks independently access the same shared file instead of using collective operations. Same-file edges connect POSIX and MPI-IO records for the same file, allowing the model to observe the pattern of independent accesses.
- **Stragglers** are slow ranks that delay the entire job. Same-rank edges help identify which files the slow rank is accessing and what patterns distinguish it from faster ranks.

[PLACEHOLDER: Attention weight visualization showing graph with edge thickness proportional to attention, highlighting imbalanced rank connections]

Figure 8: Attention visualization for load imbalance detection. Edge thickness indicates attention weight; high attention between files accessed by different ranks reveals the imbalance pattern.

Bottlenecks with local signatures. For bottlenecks that depend primarily on individual file statistics (e.g., small I/O, misalignment), the improvement from graph structure is smaller but still positive. Even for these patterns, the graph enables the model to learn which files are most important (via attention) and to share information when the same pattern appears across multiple files.

Comparison with flat aggregation. Flat-feature approaches like AIIO [10] aggregate file-level statistics to job level using functions like mean, max, and sum. This aggregation loses information about *which* files contribute to bottlenecks and *how* they relate to each other. Our graph approach preserves this information, enabling more accurate and interpretable predictions.

Figure 8 visualizes attention weights for a representative load imbalance case. The high attention between nodes representing files accessed by different ranks highlights the imbalance pattern.

5.2 Comparison with LLM-Based Approaches

IOAgent [12] and ION [11] represent a different paradigm for I/O diagnosis: using large language models to generate natural language explanations. While we do not directly compare accuracy (as IOAgent requires LLM inference and uses different evaluation protocols), we can compare the approaches along several dimensions:

Speed and scalability. IOGraphNet inference takes approximately [X] milliseconds per job, enabling real-time analysis of large log collections. LLM-based approaches require seconds to minutes per job due to LLM inference costs, making them unsuitable for analyzing millions of production logs.

Consistency. GNN predictions are deterministic given the same input; running IOGraphNet twice on the same log produces identical results. LLMs may produce varying outputs across runs due to sampling, complicating reproducibility and debugging.

Cost. IOGraphNet runs on local hardware with no API costs. LLM-based approaches using commercial APIs (e.g., GPT-4) incur significant costs at scale.

Interpretability. LLMs excel at generating natural language explanations that domain scientists can understand. IOGraphNet provides interpretability through attention weights, which indicate important file relationships but require visualization to communicate effectively.

Hybrid approaches. A promising direction is combining GNN-based detection with LLM-based explanation: IOGraphNet quickly

Table 8: Comparison of IOGraphNet with LLM-based approaches

Dimension	IOGraphNet	ION	IOAgent
Inference time	<10 ms	seconds	seconds
Consistency	Deterministic	Variable	Variable
API cost	None	High	High
Scale (logs/day)	Millions	Thousands	Thousands
Natural language	No	Yes	Yes
Domain knowledge	Learned	Prompts	RAG
Multi-label	Native	Implicit	Implicit

identifies bottleneck types and important files, then an LLM generates human-readable recommendations based on this structured output. This hybrid approach would combine the speed and accuracy of GNNs with the explanatory power of LLMs.

Table 8 summarizes these trade-offs.

5.3 Implications for Practitioners

IOGraphNet is designed for practical deployment in HPC center operations. Several features support this goal:

Continuous monitoring. The fast inference time enables integration with continuous I/O monitoring pipelines. New Darshan logs can be analyzed as they are generated, providing real-time feedback to users about I/O performance issues.

Actionable recommendations. For each detected bottleneck, we provide standard remediation advice:

- Small I/O → Increase buffer size to at least 1MB
- Missing collective → Use MPI_File_write_all instead of MPI_File_write
- Load imbalance → Redistribute I/O workload or use collective operations
- Checkpoint bottleneck → Use asynchronous I/O or compress checkpoints

Attention weights add specificity by identifying which files exhibit each problem.

User-facing integration. IOGraphNet can power user-facing tools: a command-line interface for individual log analysis, a web dashboard for aggregate statistics, or integration with job schedulers to provide post-job I/O reports.

5.4 Limitations and Future Work

Ground truth from benchmarks. Our ground-truth labels come from controlled benchmarks rather than production workloads. While benchmarks capture known I/O patterns, they may not represent all real-world bottleneck combinations. We mitigate this by including diverse benchmarks (IOR, mdtest, DLIO) covering both traditional HPC and AI/ML patterns. Future work could incorporate expert annotations on production logs for more representative training data.

System-level bottlenecks. Bottlenecks like contention and OST imbalance depend on factors external to the application (other jobs sharing storage, storage server health) that are not fully captured in Darshan logs. Integrating system-level telemetry (Lustre server statistics, network congestion metrics) could improve detection of these patterns.

Generalization across systems. Our evaluation focuses on ALCF Polaris with Lustre storage. Performance on other systems (e.g., GPFS-based, different Lustre configurations) may differ. Transfer learning techniques could enable adaptation to new systems with minimal retraining.

Temporal patterns. Our current graph construction treats files as static entities. Future work could incorporate temporal edges based on DXT traces [30] to capture phase-based I/O behavior and detect bottlenecks that emerge only during specific application phases.

6 Additional Related Work

I/O Performance Analysis. Beyond Section 2's tools, other approaches provide complementary capabilities. UMAMI [21] proposes holistic analysis of "I/O climate" and "I/O weather." Zoom-in analysis [5] combines aggregate statistics with detailed trace examination to identify root causes of performance issues. DXT Explorer [3] provides interactive visualization of fine-grained DXT traces. Recent surveys [25] comprehensively characterize I/O access patterns in HPC applications, informing bottleneck taxonomies like ours.

GNNs for System Performance. Graph neural networks have been applied to various system performance problems beyond I/O. GNN-RL [13] uses graph representations for HPC job scheduling on heterogeneous clusters. HPCGCN [32] predicts system events from cluster log data using graph convolutional networks. GNNs have also been applied to microservice latency prediction [29], modeling dependencies between services as a graph. Our work extends this line of research to HPC I/O analysis, a domain where the relational structure of file access has not previously been exploited.

Multi-Level I/O Tracing. Recorder [16] provides multi-level I/O tracing capturing operations at POSIX, MPI-IO, and HDF5 layers with minimal overhead. Combined with Darshan's aggregate statistics, such detailed traces could enable even richer graph representations.

AutoML for HPC. Automated machine learning has been applied to HPC performance optimization [28], typically for parameter tuning (e.g., I/O buffer sizes, stripe counts) rather than bottleneck detection. IOGraphNet is complementary: it identifies *what* to optimize, while AutoML tools optimize *how*.

7 Conclusion

We presented IOGraphNet, the first graph neural network approach for I/O bottleneck detection in HPC systems. Our key insight is that I/O bottlenecks often emerge from *relationships*—load imbalance across MPI ranks, missing collective operations on shared files, cross-module inefficiencies—that flat-feature models cannot capture effectively. By modeling the relational structure of I/O behavior as a graph where nodes represent file access records and edges encode relationships, IOGraphNet learns to aggregate information across related files to detect these patterns.

Our evaluation on over 2,000 benchmark logs with ground-truth labels demonstrates that IOGraphNet achieves [X]% Micro-F1, outperforming XGBoost, Random Forest, and MLP baselines. Ablation

studies confirm that the graph structure contributes [Y]% improvement over equivalent models without edges, with same-rank edges providing the largest benefit for detecting load imbalance patterns. Generalization analysis on 1.37 million production logs from ALCF Polaris shows that predictions are consistent with the expected workload mix.

IOGraphNet enables automated, accurate I/O diagnosis at scale, helping domain scientists identify and address performance bottlenecks without requiring deep I/O expertise. The fast inference time (milliseconds per job) makes real-time analysis practical, and attention weights provide interpretable explanations of which file relationships contribute to each prediction.

Future work includes three directions: (1) integration with LLMs for natural language explanations that combine IOGraphNet's accurate detection with intuitive recommendations, (2) extension to temporal graphs using DXT traces to capture phase-based I/O behavior, and (3) deployment as a production service providing real-time I/O diagnosis feedback to users of leadership-class supercomputers.

References

[1] Argonne Leadership Computing Facility. 2024. Polaris Supercomputer. https://www.alcf.anl.gov/polaris.

[2] Hammad Ather, Jean Luca Bez, Boyana Norris, and Suren Byna. 2023. Illuminating the I/O Optimization Path of Scientific Applications. In *ISC High Performance 2023 (Lecture Notes in Computer Science, Vol. 13948)*. Springer. doi:10.1007/978-3-031-32041-5_2

[3] Jean Luca Bez and Suren Byna. 2021. DXT Explorer: An Interactive Tool for Exploring I/O Traces. https://github.com/hpc-io/dxt-explorer.

[4] Jean Luca Bez, Suren Byna, and Andreas Herten. 2021. I/O Bottleneck Detection and Tuning: Connecting the Dots using Interactive Log Analysis. In *Proceedings of the 6th International Parallel Data Systems Workshop (PDSW '21)*. IEEE. doi:10.1109/PDSW54622.2021.00010

[5] Suren Byna et al. 2019. A Zoom-in Analysis of I/O Logs to Detect Root Causes of I/O Performance Bottlenecks. In *Proceedings of the 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid '19)*. IEEE, 102–111. doi:10.1109/CCGRID.2019.00021

[6] Philip Carns, Kevin Harms, William Allcock, Charles Bacon, Samuel Lang, Robert Latham, and Robert Ross. 2011. Understanding and Improving Computational Science Storage Access through Continuous Characterization. *ACM Transactions on Storage (TOS)* 7, 3, 1–26. doi:10.1145/2027066.2027068

[7] Philip Carns, Robert Latham, Robert Ross, Kamil Iskra, Samuel Lang, and Katherine Riley. 2009. 24/7 Characterization of Petascale I/O Workloads. In *Proceedings of the 2009 IEEE International Conference on Cluster Computing*. IEEE, 1–10. doi:10.1109/CLUSTR.2009.5289150

[8] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 785–794. doi:10.1145/2939672.2939785

[9] Hariharan Devarajan, Huihuo Zheng, Anthony Kougkas, Xian-He Sun, and Venkatram Vishwanath. 2021. DLIO: A Data-Centric Benchmark for Scientific Deep Learning Applications. In *Proceedings of the 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 81–91. doi:10.1109/CCGrid51090.2021.00018

[10] Bin Dong, Jean Luca Bez, and Suren Byna. 2023. AIIO: Using Artificial Intelligence for Job-Level and Automatic I/O Performance Bottleneck Diagnosis. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '23)*. ACM, 155–167. doi:10.1145/3588195.3592986

[11] Chris Egersdoerfer, Arnav Sareen, Jean Luca Bez, Suren Byna, and Dong Dai. 2024. ION: Navigating the HPC I/O Optimization Journey using Large Language Models. In *Proceedings of the 16th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage '24)*. ACM, 8–14. doi:10.1145/3655038.3665950

[12] Chris Egersdoerfer, Arnav Sareen, Jean Luca Bez, Suren Byna, Dingwen Xu, and Dong Dai. 2025. IOAgent: Democratizing Trustworthy HPC I/O Performance Diagnosis Capability via LLMs. In *Proceedings of the 39th IEEE International Parallel and Distributed Processing Symposium (IPDPS '25)*. IEEE. doi:10.1109/IPDPS57955.2025.00069

[13] Zhiying Fan et al. 2024. GNN-RL: Graph Neural Network with Reinforcement Learning for HPC Job Scheduling. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '24)*. ACM/IEEE.

[14] Matthias Fey and Jan Eric Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

[15] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems 30 (NeurIPS '17)*. 1024–1034.

[16] Stephen Herbein et al. 2020. Recorder 2.0: Multi-Level I/O Tracing. In *Proceedings of the 34th IEEE International Parallel and Distributed Processing Symposium (IPDPS '20)*. IEEE, 1037–1046. doi:10.1109/IPDPS47924.2020.00109

[17] HPC IO Consortium. 2023. IOR: Parallel Filesystem I/O Benchmark. https://github.com/hpc/ior.

[18] Thomas N. Kipf and Max Welling. 2016. Variational Graph Auto-Encoders. *NIPS Workshop on Bayesian Deep Learning* (2016). arXiv:1611.07308

[19] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *Proceedings of the 5th International Conference on Learning Representations (ICLR '17)*.

[20] Jintang Li, Ruofan Wu, Wangbin Sun, Liang Chen, Sheng Tian, Liang Zhu, Changhua Meng, Zibin Zheng, and Weiqiang Wang. 2023. What's Behind the Mask: Understanding Masked Graph Modeling for Graph Autoencoders. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*. ACM, 1268–1279. doi:10.1145/3580305.3599546

[21] Glenn K. Lockwood, Wucherl Yoo, Suren Byna, Nicholas J. Wright, Shane Snyder, Kevin Harms, Zachary Nault, and Philip Carns. 2017. UMAMI: A Recipe for Generating Meaningful Metrics through Holistic I/O Performance Analysis. In *Proceedings of the 2nd Joint International Workshop on Parallel Data Storage and Data Intensive Scalable Computing Systems (PDSW-DISCS '17)*. ACM, 55–60. doi:10.1145/3149393.3149395

[22] Jay Lofstead et al. 2020. HPC I/O Throughput Bottleneck Analysis with Explainable Local Models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20)*. IEEE/ACM. doi:10.1109/SC41405.2020.00068

[23] Scott M. Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS '17)*. 4765–4774.

[24] Huong Luu, Marianne Winslett, William Gropp, Robert Ross, Philip Carns, Kevin Harms, Mr Prabhat, Suren Byna, and Yushu Yao. 2015. A Multiplatform Study of I/O Behavior on Petascale Supercomputers. In *Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '15)*. ACM, 33–44. doi:10.1145/2749246.2749269

[25] Tirthak Patel, Suren Byna, Glenn K. Lockwood, Nicholas J. Wright, Philip Carns, and Robert Ross. 2024. I/O Access Patterns in HPC Applications: A 360-Degree Survey. *Comput. Surveys* 56, 5, 1–37. doi:10.1145/3611007

[26] Shane Snyder, Philip Carns, Kevin Harms, Robert Ross, Glenn K. Lockwood, and Nicholas J. Wright. 2016. Modular HPC I/O Characterization with Darshan. In *Proceedings of the 5th Workshop on Extreme-Scale Programming Tools (ESPT '16)*. IEEE, 9–17. doi:10.1109/ESPT.2016.9

[27] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *Proceedings of the 6th International Conference on Learning Representations (ICLR '18)*.

[28] Xingfu Wu, William Kramer, et al. 2022. Autotuning for HPC Applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE.

[29] Zhongzhi Wu et al. 2022. Graph Neural Networks for Microservice Performance Prediction. In *Proceedings of the ACM Web Conference*. ACM.

[30] Huihuo Xu, Shane Snyder, Omkar Kulkarni, Venkat Venkatesan, Philip Carns, Suren Byna, Rob Sisneros, and Kishore Chadalavada. 2017. DXT: Darshan eXtended Tracing. In *Cray User Group Conference (CUG '17)*.

[31] Izzet Yildirim, Hariharan Devarajan, Anthony Kougkas, and Xian-He Sun. 2025. WisIO: Automated I/O Bottleneck Detection with Multi-Perspective Views for HPC Workflows. In *Proceedings of the 2025 International Conference on Supercomputing (ICS '25)*. ACM. doi:10.1145/3650200.3656632

[32] Xiongchao Zeng et al. 2023. HPCGCN: A Predictive Framework on High Performance Computing Cluster Log Data Using Graph Convolutional Networks. *Sensors* 23, 3 (2023), 1493. doi:10.3390/s23031493

A Extended Experimental Results

A.1 Hyperparameter Search

Table 9 reports the hyperparameter search results. We searched over learning rate, hidden dimensions, number of attention heads, and dropout rate using grid search with 5-fold cross-validation on the training set. The best configuration uses learning rate 0.001, 64 hidden dimensions, 4 attention heads, and dropout 0.2.

Table 9: Hyperparameter search results (Micro-F1 on validation)

LR	Hidden	Heads	Drop.	F1
0.01	64	4	0.2	[PLACEHOLDER: 0.82]
0.001	32	4	0.2	[PLACEHOLDER: 0.84]
0.001	**64**	**4**	**0.2**	[PLACEHOLDER: **0.86**]
0.001	128	4	0.2	[PLACEHOLDER: 0.85]
0.001	64	2	0.2	[PLACEHOLDER: 0.84]
0.001	64	8	0.2	[PLACEHOLDER: 0.85]
0.001	64	4	0.1	[PLACEHOLDER: 0.84]
0.001	64	4	0.3	[PLACEHOLDER: 0.85]
0.0001	64	4	0.2	[PLACEHOLDER: 0.83]

A.2 Additional Ablation Studies

Number of GAT Layers. Table 10 shows the effect of varying the number of GAT layers. Two layers provide the best trade-off between expressiveness and overfitting. Deeper networks (3–4 layers) show slight degradation, likely due to over-smoothing where node representations become indistinguishable.

Table 10: Effect of number of GAT layers

Layers	Micro-F1	Training Time
1	[PLACEHOLDER: 0.82]	[PLACEHOLDER: 15] min
2	[PLACEHOLDER: **0.86**]	[PLACEHOLDER: 20] min
3	[PLACEHOLDER: 0.85]	[PLACEHOLDER: 28] min
4	[PLACEHOLDER: 0.84]	[PLACEHOLDER: 35] min

Graph Pooling Methods. We compare mean pooling with alternative aggregation strategies (Table 11). Mean pooling slightly outperforms max and sum pooling. Attention-based pooling (Set2Set) provides marginal improvement but at higher computational cost.

Table 11: Effect of graph pooling method

Pooling	Micro-F1	Inference Time
Mean	[PLACEHOLDER: **0.86**]	[PLACEHOLDER: 5] ms
Max	[PLACEHOLDER: 0.84]	[PLACEHOLDER: 5] ms
Sum	[PLACEHOLDER: 0.83]	[PLACEHOLDER: 5] ms
Set2Set	[PLACEHOLDER: 0.87]	[PLACEHOLDER: 12] ms

B Dataset Details

B.1 Polaris System Characteristics

ALCF Polaris is a 560-node HPE Apollo Gen10+ system with:

- **Compute:** AMD EPYC Milan CPUs (32 cores), 4× NVIDIA A100 GPUs per node
- **Memory:** 512 GB DDR4 RAM, 160 GB HBM2e GPU memory per node
- **Storage:** Eagle Lustre file system (100 PB, 650 GB/s peak bandwidth)
- **Network:** HPE Slingshot interconnect (200 Gb/s per node)

B.2 Polaris Log Statistics

Table 12 provides detailed statistics for the Polaris production dataset.

Table 12: Polaris production dataset statistics

Statistic	Value
Total logs	1,378,117
Date range	Apr 2024 – Jan 2026
Unique users	[PLACEHOLDER: X]
Unique projects	[PLACEHOLDER: Y]
Logs with POSIX module	[PLACEHOLDER: X%]
Logs with MPI-IO module	[PLACEHOLDER: Y%]
Logs with STDIO module	[PLACEHOLDER: Z%]
Logs with HDF5 module	[PLACEHOLDER: W%]
Median files per job	[PLACEHOLDER: 12]
Mean files per job	[PLACEHOLDER: 45]
Max files per job	[PLACEHOLDER: 10,000+]
Median job duration	[PLACEHOLDER: X] min
Median I/O bytes	[PLACEHOLDER: Y] GB

B.3 Benchmark Configuration Details

Table 13 details the benchmark configurations used to generate ground-truth labels.

B.4 Feature Normalization

All node features are normalized using z-score standardization computed on the training set:

$$x'_i = \frac{x_i - \mu_i}{\sigma_i + \epsilon} \tag{11}$$

where μ_i and σ_i are the mean and standard deviation of feature i across all training samples, and $\epsilon = 10^{-8}$ prevents division by zero.

For features with heavy-tailed distributions (e.g., byte counts, operation counts), we apply log transformation before normalization:

$$x'_i = \frac{\log(1 + x_i) - \mu_i}{\sigma_i + \epsilon} \tag{12}$$

C Implementation Details

C.1 Structure-Aware Baseline Features

To ensure fair comparison between IOGraphNet and flat-feature baselines, we augment XGBoost, Random Forest, and MLP with graph-derived statistical features. Table 14 lists these features, which allow baselines to access summarized structural information without the graph itself.

Key Insight: If IOGraphNet outperforms structure-aware baselines, it proves that *message-passing* (local neighborhood aggregation) provides value beyond statistical summaries. The GNN can identify *which specific node* is the outlier, while baselines only see "average degree = 5."

C.2 Feature List

Table 15 lists all features extracted from Darshan logs for each file node.

Table 13: Benchmark configurations for ground-truth label generation (8 bottleneck categories)

Bottleneck Type	Benchmark	Configuration	Label Criteria
Metadata Storm	mdtest	10K files/proc, stat-heavy	Meta time > data time, opens/ops >10
Small I/O	IOR	`-t 64 -b 64` (64-byte transfers)	>80% ops <1KB
Data Imbalance	Custom MPI	Rank 0 writes 10× more; sleep injection	CV(bytes) > 0.5, max/median time > 2
Write Saturation	IOR	Large sequential writes, high volume	High bytes, low bandwidth vs peak
Read Saturation	IOR	Large sequential reads, high volume	High bytes, low bandwidth vs peak
Interface Misuse	IOR	POSIX mode on shared file	Shared file, high indep ops, zero collective
File-per-Process	mdtest/Custom	1000+ files created per rank	Files created > 100 × ranks
Healthy	IOR	Optimized collective I/O	High bandwidth, balanced access

Table 14: Structure-aware features for baseline models

Category	Feature	Topological Intuition
Global	Total Ops (Read/Write/Meta)	Overall job intensity (AIIO-style)
	Total Bytes (R/W)	Data volume
Graph (Node)	Max/Mean Rank Degree	Fan-out: metadata stress
	Max/Mean File Degree	Fan-in: contention/locking
	Degree Std. Dev.	Load imbalance across nodes
Graph (Edge)	Max Concurrency Weight	Peak contention on single file
	Mean Concurrency Weight	General system pressure
	Edge Weight Skewness	Stragglers vs. uniform I/O
Imbalance	Gini Coefficient (bytes)	Data distribution inequality
	Max/Mean Time Ratio	Straggler detection

Table 15: Node features extracted from Darshan logs (partial list)

Category	Feature	Description
Operations	read_count	Number of read operations
	write_count	Number of write operations
	bytes_read	Total bytes read
	bytes_written	Total bytes written
	seq_reads	Sequential read count
	seq_writes	Sequential write count
Access Size	size_0_100	Ops with size 0–100B
	size_100_1K	Ops with size 100B–1KB
	size_1K_10K	Ops with size 1KB–10KB
	...	...
	size_1G_plus	Ops with size >1GB
Timing	read_time	Total read time (s)
	write_time	Total write time (s)
	meta_time	Metadata operation time
	io_time	Total I/O time
Derived	avg_read_size	bytes_read / read_count
	bandwidth	bytes / time
	meta_ratio	meta_time / io_time

C.3 Graph Construction Algorithm

Algorithm 1 describes the graph construction procedure.

C.4 Model Hyperparameters

Table 16 lists all model hyperparameters with justifications.

Algorithm 1 Graph Construction from Darshan Log

Require: Darshan log L with file records $\{r_1, ..., r_n\}$
Ensure: Graph $G = (V, E, X)$

```
Initialize V ← ∅, E ← ∅
for each record r_i ∈ L do
    Add node v_i to V
    Extract features x_i from r_i
end for
// Add same-rank edges
for each pair (v_i, v_j) where i < j do
    if rank(r_i) = rank(r_j) then
        Add edge (v_i, v_j) to E
    end if
end for
// Add same-file edges
for each pair (v_i, v_j) where i < j do
    if filename(r_i) = filename(r_j) then
        Add edge (v_i, v_j) to E
    end if
end for
// Compute edge weights (concurrency)
for each edge (v_i, v_j) ∈ E do
    w_ij ← IoU(time_window(r_i), time_window(r_j))
end for
// Fallback for sparse graphs
if |E| < 1 then
    Add edges between all node pairs
end if
Normalize features: X ← normalize([x_1, ..., x_n])
return G = (V, E, X)
```

D Reproducibility

D.1 Code and Data Availability

Upon acceptance, we will release:

- **Code:** Complete implementation including data processing, model training, and evaluation scripts (GitHub repository, MIT license)
- **Benchmark dataset:** Darshan logs from IOR, mdtest, and DLIO with ground-truth labels
- **Trained models:** Pre-trained IOGraphNet weights for inference

Table 16: IOGraphNet hyperparameters

Parameter	Value	Justification
GAT layers	2	Avoids over-smoothing
Hidden dim	64	Balance capacity/speed
Attention heads	4	Multi-perspective attention
Dropout	0.2	Regularization
Learning rate	0.001	Standard for Adam
Weight decay	10^{-5}	L2 regularization
Batch size	32	GPU memory constraint
Max epochs	100	With early stopping
Early stop patience	10	Prevent overfitting
Activation	ELU	Smooth gradients
Pooling	Mean	Permutation invariant

- **Documentation:** Installation guide, usage examples, and API documentation

The Polaris production logs cannot be released due to ALCF data policies, but we provide detailed statistics (Appendix B) to enable replication on other systems.

D.2 Hardware Specifications

Experiments were conducted on:

- **Training:** Single NVIDIA A100 (80 GB HBM2e)
- **CPU:** AMD EPYC 7763 (64 cores)
- **Memory:** 512 GB DDR4
- **Software:** Python 3.10, PyTorch 2.0, PyTorch Geometric 2.3

D.3 Training Procedure

(1) **Data preparation:** Parse Darshan logs, extract features, construct graphs
(2) **Split:** 70% train, 15% validation, 15% test (stratified by label distribution)
(3) **Training:** Adam optimizer, batch size 32, max 100 epochs
(4) **Early stopping:** Monitor validation Micro-F1, patience 10 epochs
(5) **Model selection:** Checkpoint with best validation Micro-F1
(6) **Evaluation:** Final metrics on held-out test set

Total training time: approximately [PLACEHOLDER: 2] hours on A100 GPU.

D.4 Random Seeds

For reproducibility, we fix random seeds:

- Python random: 42
- NumPy: 42
- PyTorch: 42
- CUDA: deterministic mode enabled

We report mean and standard deviation over 5 runs with different seeds in Table 3.